

CSE 4303

Data Structures

Topic: Linked Lists

Sabbir Ahmed

Assistant Prof | CSE | IUT

sabbirahmed@iut-dhaka.edu

Recap lists...

Collection of elements of the same datatype

Operations: read, update, insert

We can easily satisfy these requirements using an array!

Recap lists...

Collection of elements of the same datatype

Operations: read $O(1)$, update $O(1)$, insert $O(1)$

We can easily satisfy these requirements using an array!

Recap lists...

Collection of elements of the same datatype

Operations: read $O(1)$, update $O(1)$, insert $O(1)$

*Can Insert only at the back
(If space is available!)*

We can easily satisfy these requirements using an array!

Recap lists...

Collection of elements of the same datatype

Operations: read $O(1)$, update $O(1)$, insert $O(1)$

Can we introduce a dynamic list without any fixed capacity?

*Can Insert only at the back
(If space is available!)*

Our **dynamic list**'s requirements (can vary in different scenarios)...

Collection of elements of the same datatype

Operations:

read or update item

insert item in begin, any position, end

remove item from begin, any position, end

We can also satisfy these requirements using an array!

Our dynamic list's requirements (can vary in different scenarios)...

Collection of elements of the same datatype

Operations:

read or update item

insert item in begin, any position, end

remove item from begin, any position, end

We can also satisfy these requirements using an array!

Our dynamic list's requirements (can vary in different scenarios)...

Collection of elements of the same datatype

Operations:

read $O(1)$ or update item $O(1)$

insert item in begin, any position, end $O(1)$

remove item from begin, any position, end

We can also satisfy these requirements using an array!

Our dynamic list's requirements (can vary in different scenarios)...

Collection of elements of the same datatype

Operations:

read $O(1)$ or update item $O(1)$

insert item in begin $O(?)$, any position $O(?)$, end $O(1)$

remove item from begin, any position, end

We can also satisfy these requirements using an array!

Our dynamic list's requirements (can vary in different scenarios)...

Collection of elements of the same datatype

Operations:

read $O(1)$ or update item $O(1)$

insert item in $\boxed{\text{begin } O(N)}$, $\boxed{\text{any position } O(N)}$, end $O(1)$

remove item from begin, any position, end

Need to shift items one step to the right to make a vacant space!

We can also satisfy these requirements using an array!

Our dynamic list's requirements (can vary in different scenarios)...

Collection of elements of the same datatype

Operations:

read $O(1)$ or update item $O(1)$

insert item in begin $O(N)$, any position $O(N)$, end $O(1)$

remove item from begin, any position, end $O(?)$

We can also satisfy these requirements using an array!

Our dynamic list's requirements (can vary in different scenarios)...

Collection of elements of the same datatype

Operations:

read $O(1)$ or update item $O(1)$

insert item in begin $O(N)$, any position $O(N)$, end $O(1)$

remove item from begin, any position, end $O(1)$

Simply keep track of the last index!

We can also satisfy these requirements using an array!

Our dynamic list's requirements (can vary in different scenarios)...

Collection of elements of the same datatype

Operations:

read $O(1)$ or update item $O(1)$

insert item in begin $O(N)$, any position $O(N)$, end $O(1)$

remove item from begin $O(?)$, any position $O(?)$,
end $O(1)$

We can also satisfy these requirements using an array!

Our dynamic list's requirements (can vary in different scenarios)...

Collection of elements of the same datatype

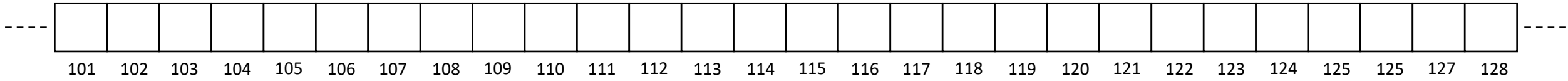
Operations:

read $O(1)$ or update item $O(1)$

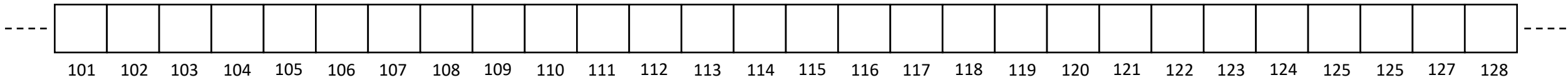
insert item in begin $O(N)$, any position $O(N)$, end $O(1)$

remove item from begin $O(N)$, any position $O(N)$,
end $O(1)$

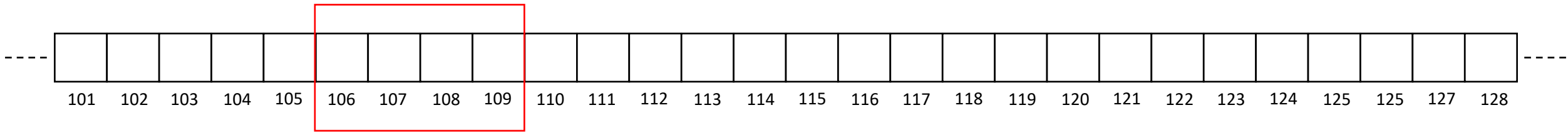
Need to shift items to left side to fill up the deleted slot!



*Continuous memory
blocks each of size
one byte*

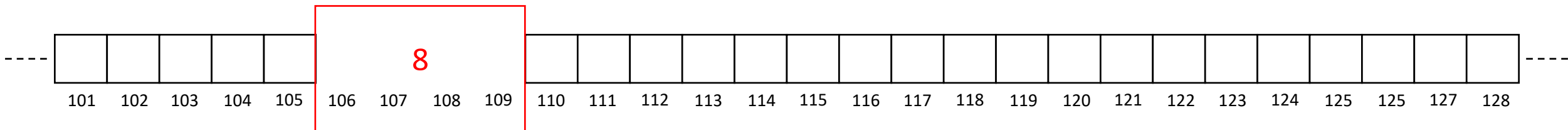


What happens for
`int x = 8 ;`



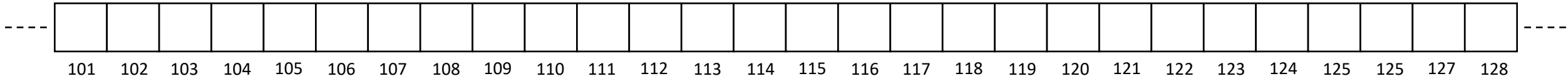
What happens for
`int x = 8 ?`

Allocate 4 bytes of memory

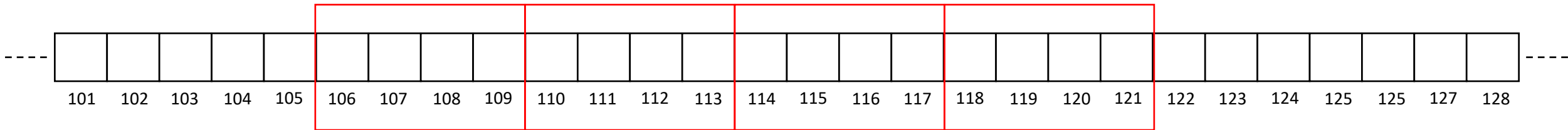


What happens for
`int x = 8 ;`

Allocate 4 bytes of memory
Then store the value

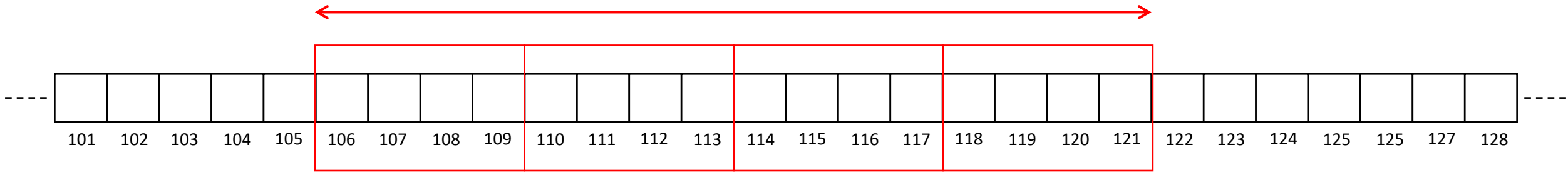


*What happens for
int arr[4] ?*



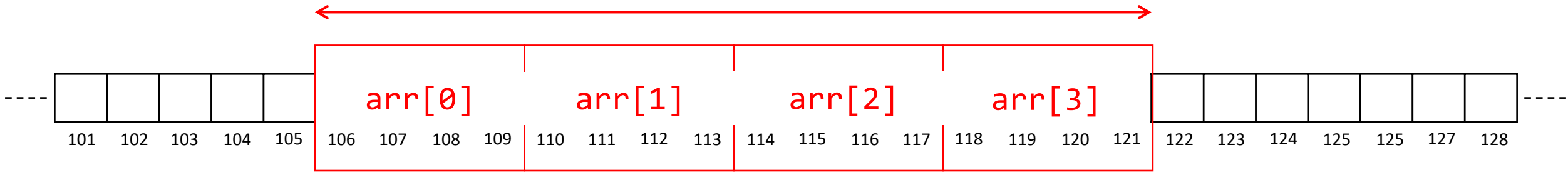
*What happens for
int arr[4] ?*

- *Allocate 4 bytes for each position of the array!*



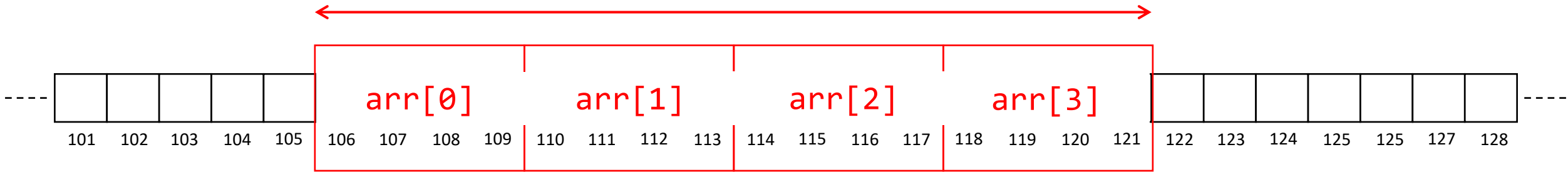
*What happens for
`int arr[4]` ?*

- *Allocate 4 bytes for each position of the array!*
- *Continuous blocks!*

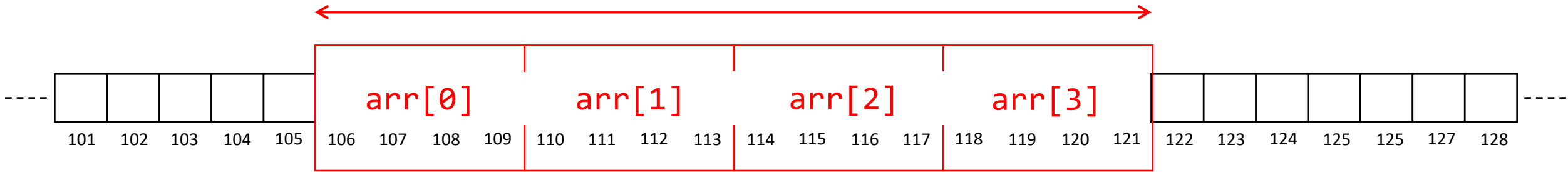


*What happens for
`int arr[4]` ?*

- *Allocate 4 bytes for each position of the array!*
- *Continuous blocks!*

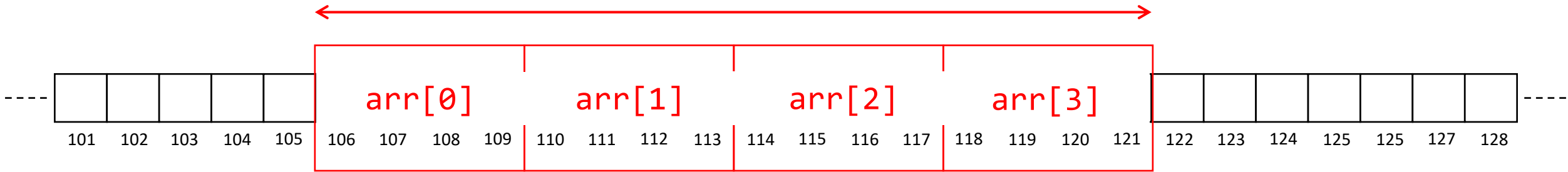


Memory location of i^{th} item
 $= \text{base} + i \times \text{byte_per_item}$



Memory location of i^{th} item
= $\text{base} + i \times \text{byte_per_item}$

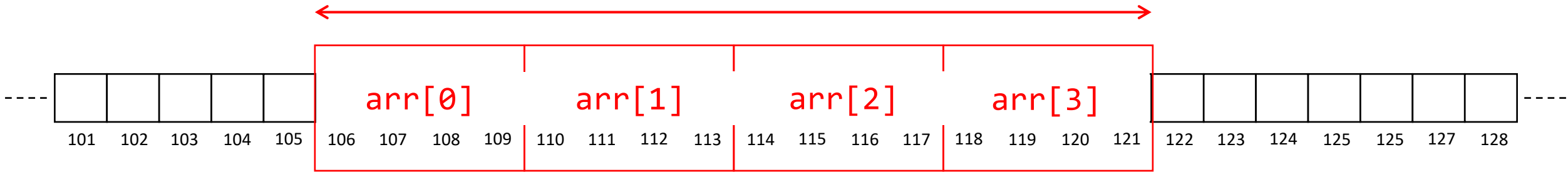
Here, $\text{base} = 106$
So the 3rd item will be at
= $106 + 2 \times 4$
= 114^{th} memory location!



Memory location of i^{th} item
= $\text{base} + i \times \text{byte_per_item}$

*Using this simple formula,
an array can provide
constant time accessing!*

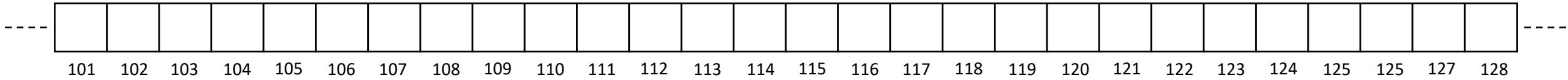
Here, $\text{base} = 106$
So the 3rd item will be at
= $106 + 2 \times 4$
= 114^{th} memory location!



Memory location of i^{th} item
= $\text{base} + i \times \text{byte_per_item}$

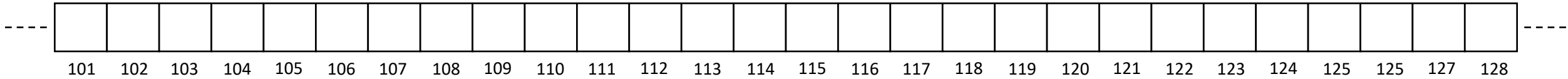
One drawback!

*Once the array is declared,
can't extend anymore 😞*



Simple solution:

Instead of occupying a large chunk of memory, allocate memory as a new item comes!

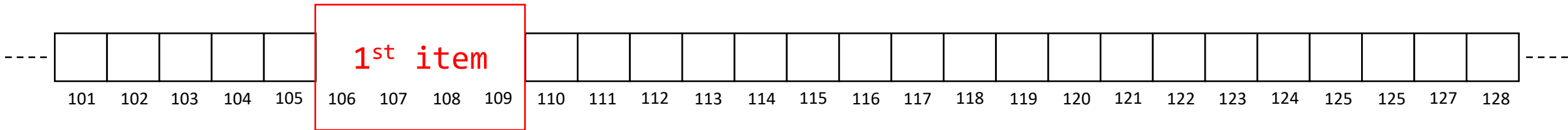


`insert(1st item):`

- Ask the memory manager for 4-byte space!

Simple solution:

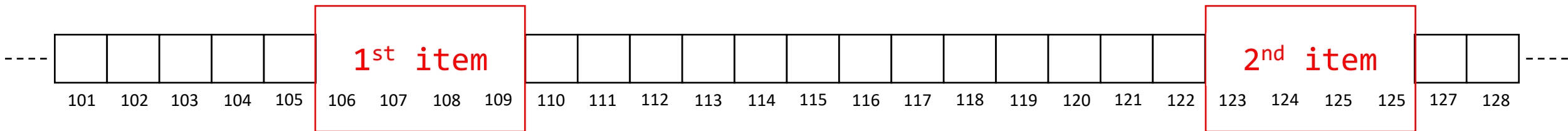
Instead of occupying a large chunk of memory, allocate memory as a new item comes!



`insert(1st item):`
- Ask the memory manager
for 4-byte space!

Simple solution:

*Instead of occupying a
large chunk of memory,
allocate memory as a new
item comes!*

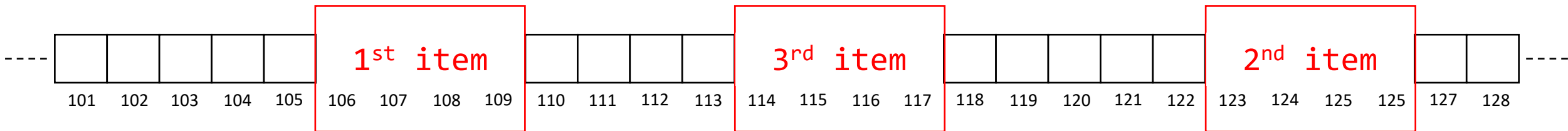


`insert(2nd item):`

- Ask the memory manager for 4-byte space again!

Simple solution:

Instead of occupying a large chunk of memory, allocate memory as a new item comes!

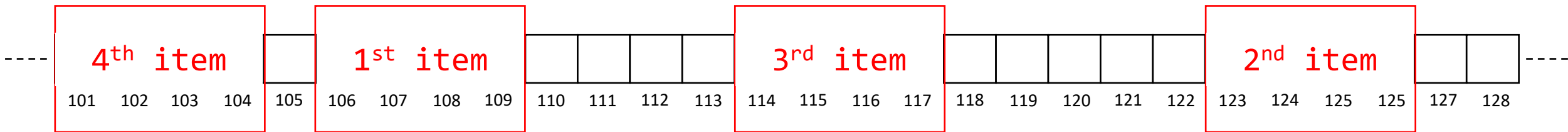


`insert(3rd item):`

- Ask the memory manager for 4-byte space again!

Simple solution:

Instead of occupying a large chunk of memory, allocate memory as a new item comes!

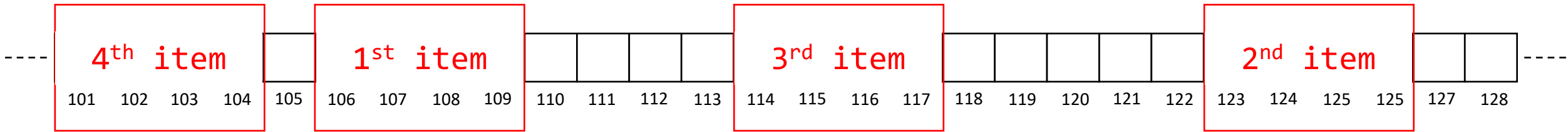


`insert(4th item):`

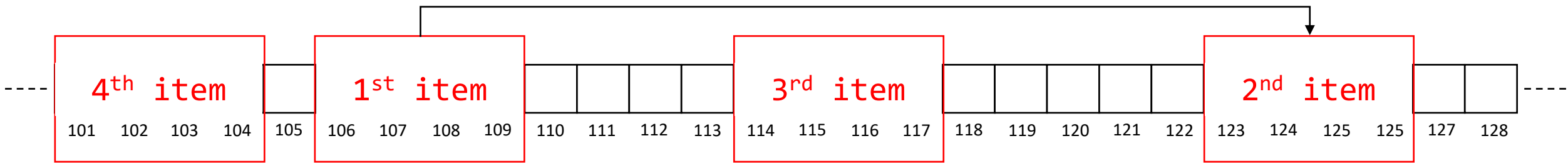
- Ask the memory manager for 4-byte space again!

Simple solution:

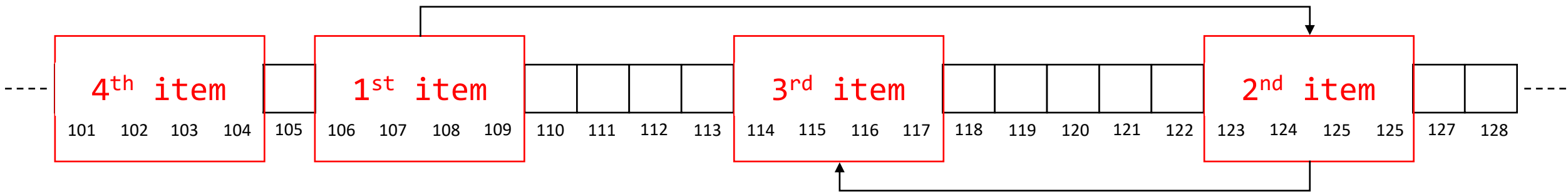
Instead of occupying a large chunk of memory, allocate memory as a new item comes!



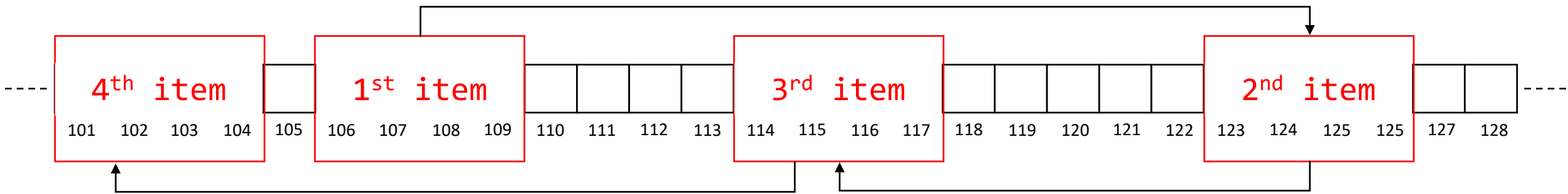
Since memory assignment isn't continuous, each item should remember its next item's address!



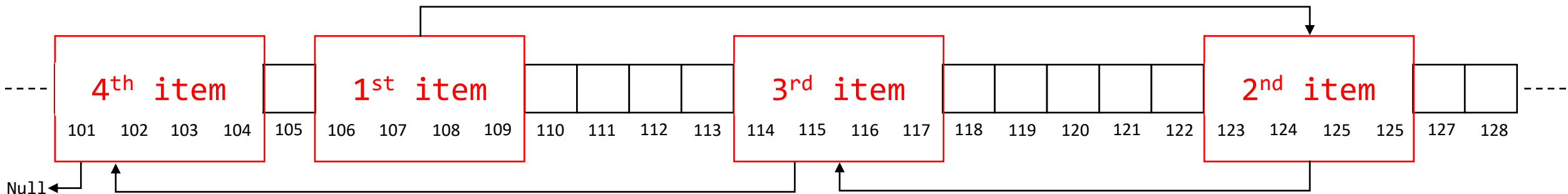
Since memory assignment isn't continuous, each item should remember its next item's address!



Since memory assignment isn't continuous, each item should remember its next item's address!



Since memory assignment isn't continuous, each item should remember it's next item's address!



Since memory assignment isn't continuous, each item should remember its next item's address!

1 st item	address of 2 nd item
----------------------	------------------------------------

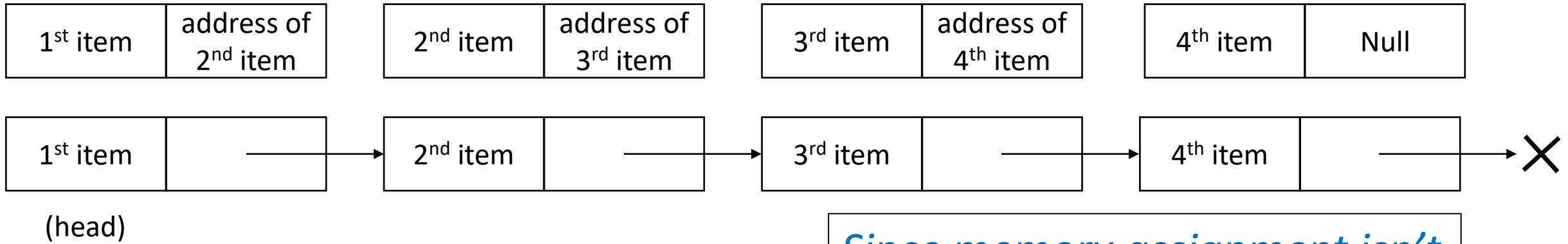
2 nd item	address of 3 rd item
----------------------	------------------------------------

3 rd item	address of 4 th item
----------------------	------------------------------------

4 th item	Null
----------------------	------

Since memory assignment isn't continuous, each item should remember its next item's address!

We need to remember the base address (head)



Since memory assignment isn't continuous, each item should remember its next item's address!

We need to remember the base address (head)

Linked lists!

Operations using linked list:

*Insert/update/delete an item from
front/back/anyPosition!*

```
Struct node{  
    int data  
    struct node* next  
}
```



```
Struct node{  
    int data  
    struct node* next  
}
```



```
Struct node* head = Null
```

*Always keeps track of the
front of LL*

```
Struct node{  
    int data  
    struct node* next  
}
```



```
Struct node* head = Null  
Struct node* tail = Null
```

*Can be used to keep track
of the back (if needed)*

```
node* create_node(int x){
```

Returns a new node with value 'x'

Allocate memory

```
node* create_node(int x){  
    struct node* newNode = new node()
```

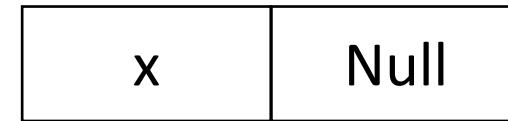
?



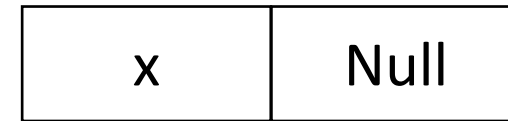
```
}
```

```
node* create_node(int x){  
    struct node* newNode = new node()  
    newNode->data = x  
    newNode->next = Null  
    ?  
}
```

Allocate memory
Assign values!



```
node* create_node(int x){  
    struct node* newNode = new node()  
    newNode->data = x  
    newNode->next = Null  
    return newNode  
}
```



```
insert_front(int x){
```

?

```
}
```

```
insert_front(int x){  
    node* newNode = create_node(x)
```

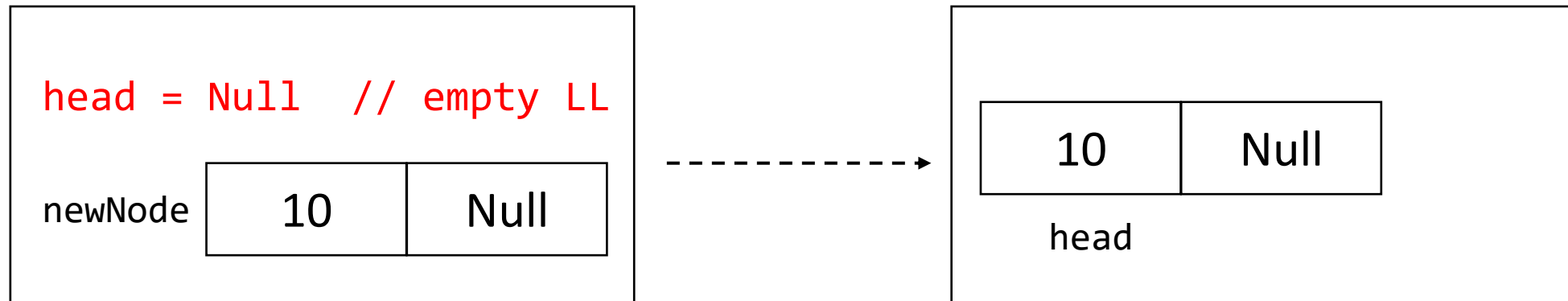
?

```
}
```

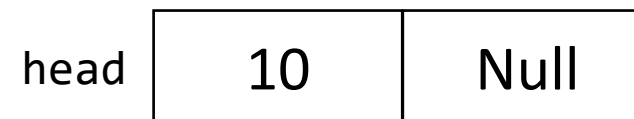
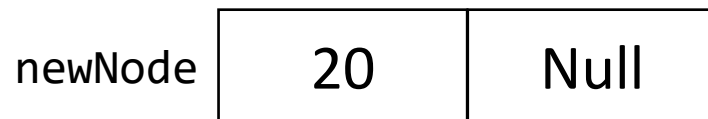
newNode

10	Null
----	------

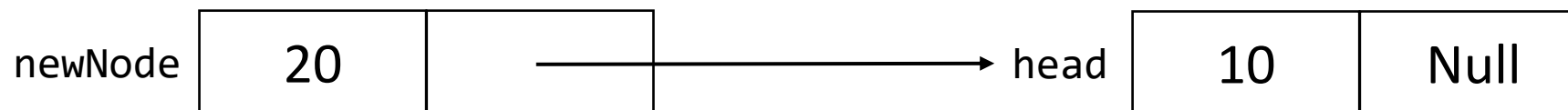
```
insert_front(int x){  
    node* newNode = create_node(x)  
    if head == Null  
        head = newNode  
  
}
```



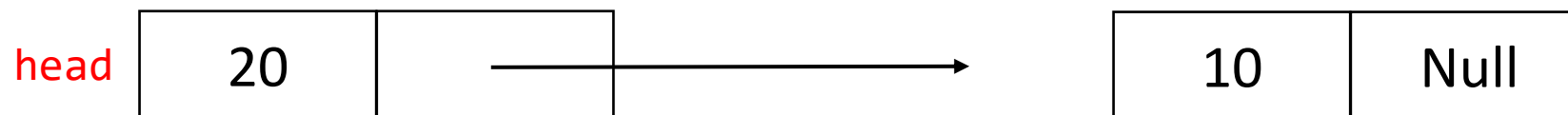
```
insert_front(int x){  
    node* newNode = create_node(x)  
    if head == Null  
        head = newNode  
    else  
  
}
```



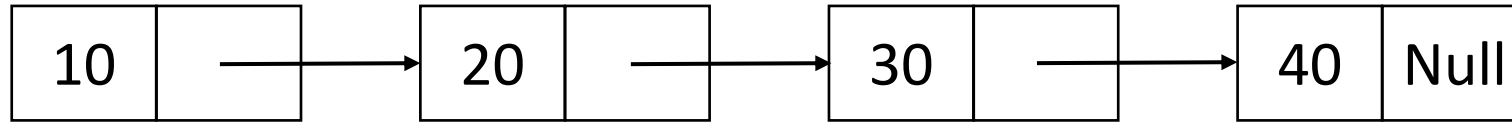
```
insert_front(int x){  
    node* newNode = create_node(x)  
    if head == Null  
        head = newNode  
    else  
        newNode->next = head  
}
```



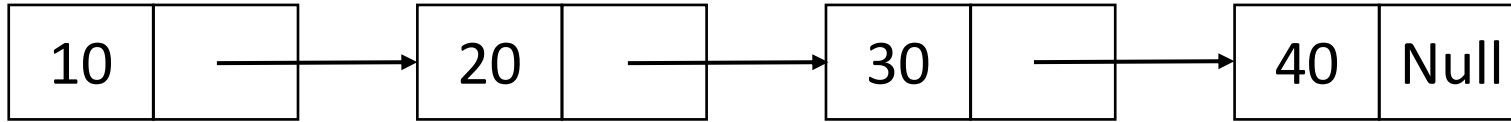
```
insert_front(int x){  
    node* newNode = create_node(x)  
    if head == Null  
        head = newNode  
    else  
        newNode->next = head  
        head = newNode  
}
```



`insert_after_position(int x, int position)`

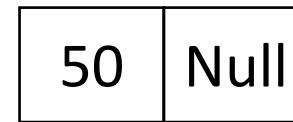
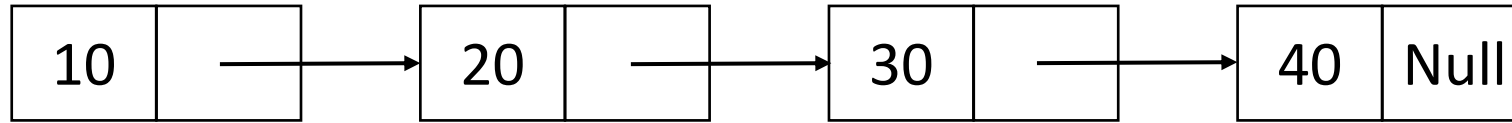


`insert_after_position(int x, int position)`



Insert 50 after the
3rd position

`insert_after_position(int x, int position)`

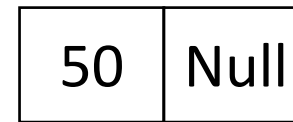
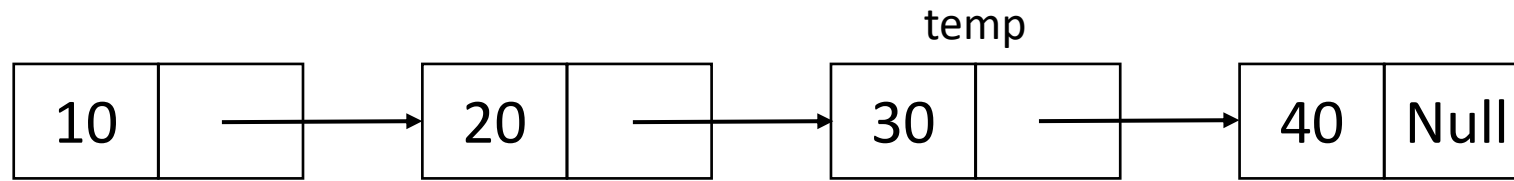


newNode

Insert 50 after the
3rd node/position

- Create a new node
with a value of 50

`insert_after_position(int x, int position)`

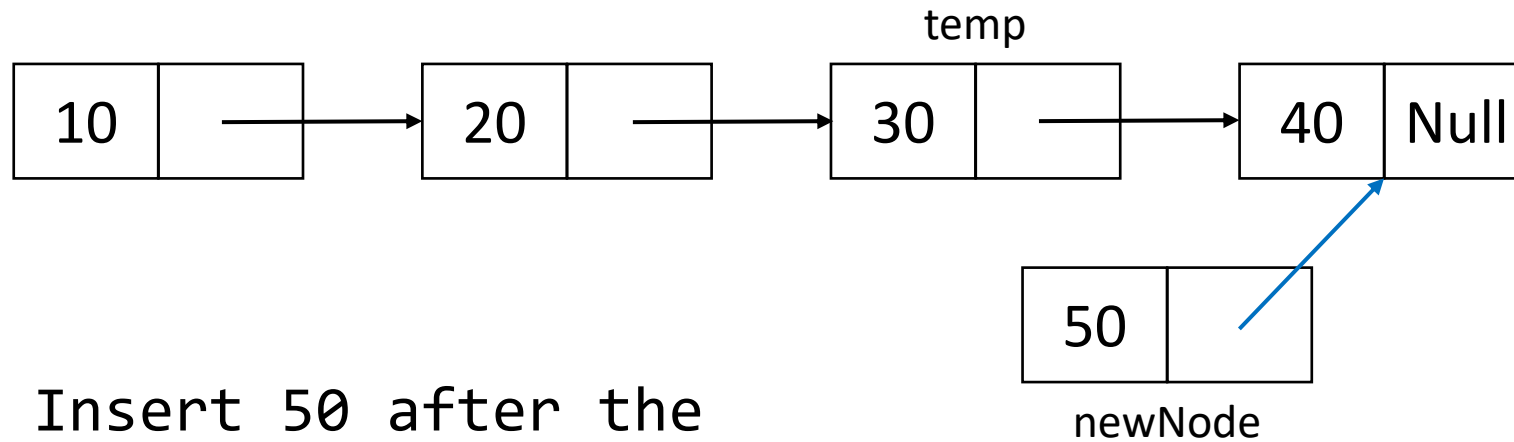


newNode

Insert 50 after the
3rd node/position

- Create a new node with a value of 50
- Find the appropriate position for insertion (Lets call it temp)

`insert_after_position(int x, int position)`

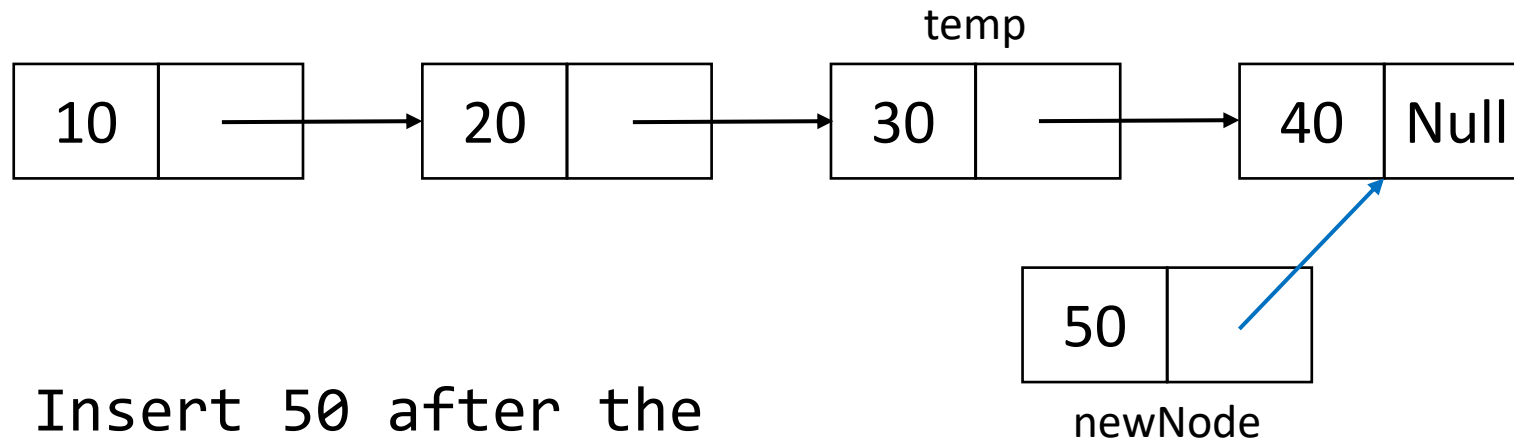


Insert 50 after the
3rd node/position

- Create a new node with a value of 50
- Find the appropriate position for insertion (Lets call it temp)

50->next = 40
? = ?

`insert_after_position(int x, int position)`

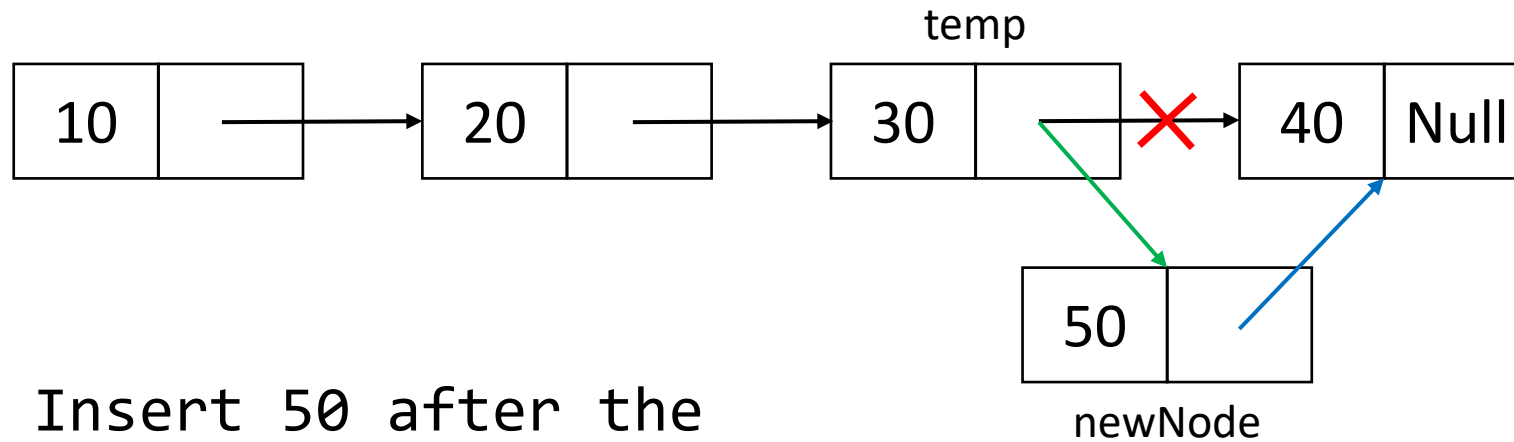


Insert 50 after the
3rd node/position

- Create a new node with a value of 50
- Find the appropriate position for insertion (Lets call it temp)

`50->next = 40`
`newNode->next = temp->next`

`insert_after_position(int x, int position)`



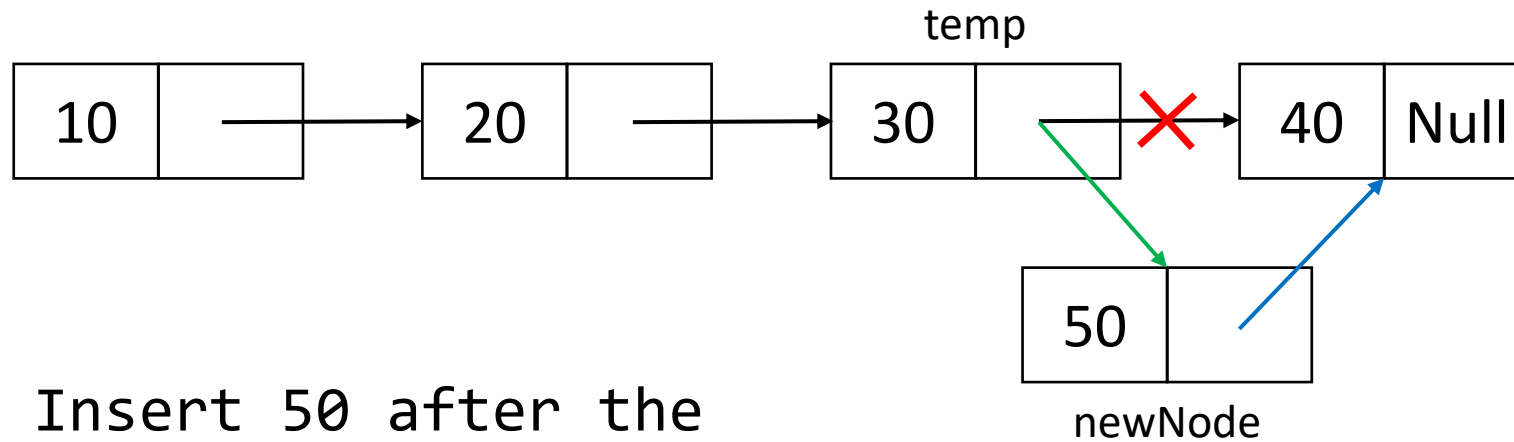
Insert 50 after the
3rd node/position

- Create a new node with a value of 50
- Find the appropriate position for insertion (Lets call it temp)

$50 \rightarrow \text{next} = 40$
 $\text{newNode} \rightarrow \text{next} = \text{temp} \rightarrow \text{next}$

$30 \rightarrow \text{next} = 50$
 $? = ?$

`insert_after_position(int x, int position)`



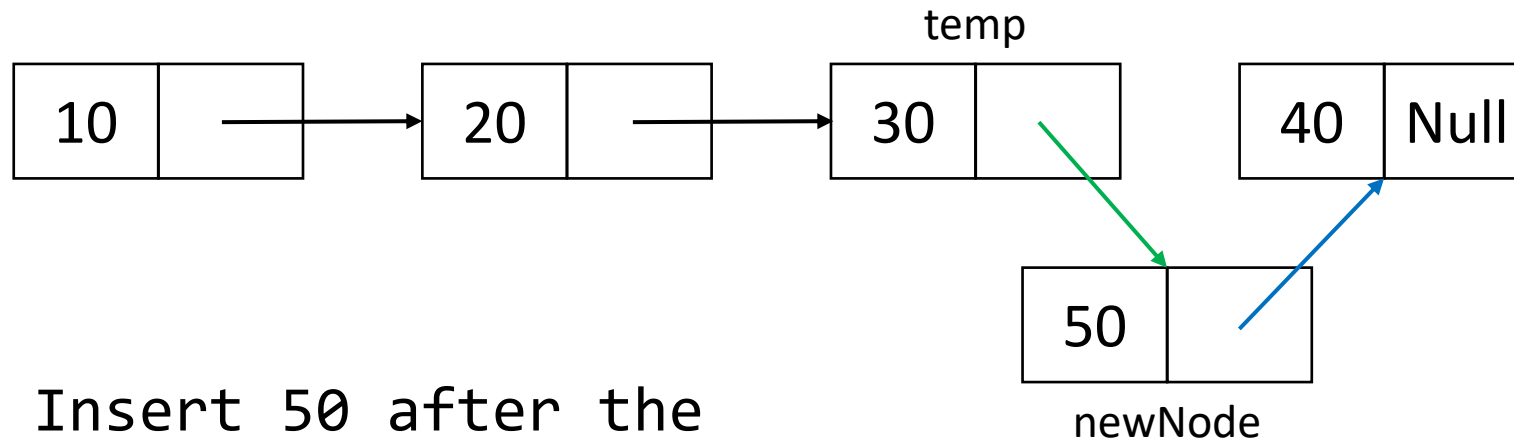
Insert 50 after the
3rd node/position

- Create a new node with a value of 50
- Find the appropriate position for insertion (Lets call it temp)

`50->next = 40`
`newNode->next = temp->next`

`30->next = 50`
`temp->next = newNode`

`insert_after_position(int x, int position)`



Insert 50 after the
3rd node/position

- Create a new node with a value of 50
- Find the appropriate position for insertion (Lets call it temp)

`newNode->next = temp->next`
`temp->next = newNode`

```
insert_after_position(int x, int position){
```

```
}
```

```
insert_after_position(int x, int position){
```

Allocate memory

check if the position is head!

Otherwise traverse till the desired position

Fix the links

```
}
```

```
insert_after_position(int x, int position){  
    node* newNode = create_node(x)
```

check if the position is head!

Otherwise traverse till the desired position

Fix the links

```
}
```

```
insert_after_position(int x, int position){  
    node* newNode = create_node(x)  
    if position == 1  
        insert_front(x)
```

Otherwise traverse till the desired position

Fix the links

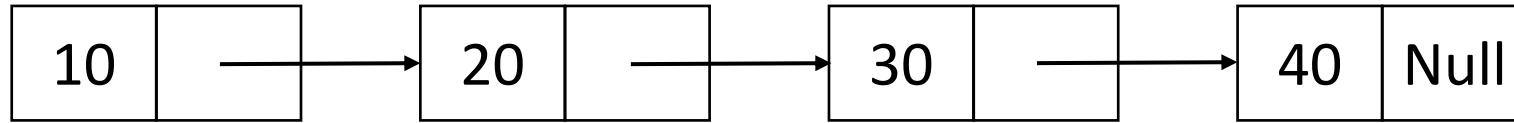
```
}
```

```
insert_after_position(int x, int position){  
    node* newNode = create_node(x)  
    if position == 1  
        insert_front(x)  
    else  
        struct node* temp = head  
        for i=1 upto position:  
            temp = temp->next  
  
    }  
}
```

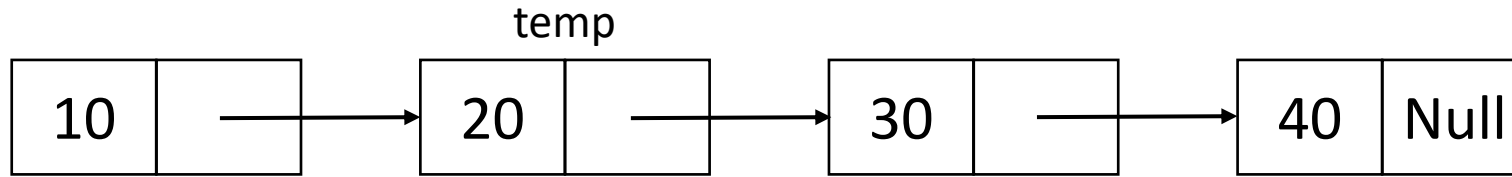
Fix the links

```
insert_after_position(int x, int position){  
    node* newNode = create_node(x)  
    if position == 1  
        insert_front(x)  
    else  
        struct node* temp = head  
        for i=1 upto position:  
            temp = temp->next  
  
        newNode->next = temp->next  
        temp->next = newNode  
}
```

`delete_value(int value)`



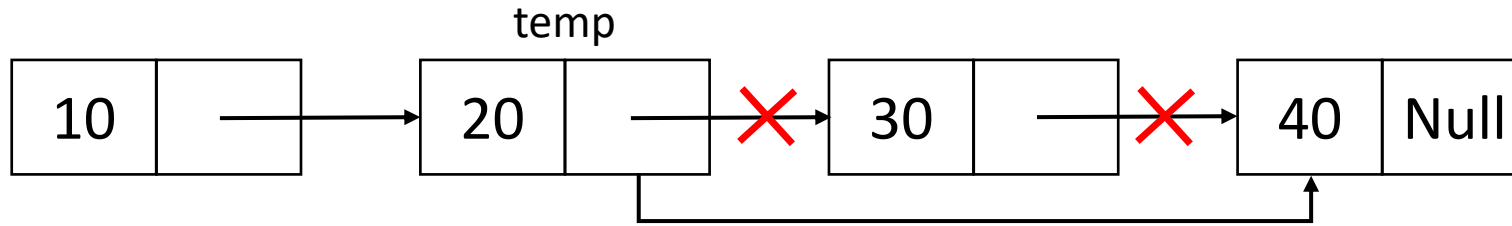
`delete_value(int value)`



Delete node with
value 30

- Find the node **for which the next node contains** the target value (Lets call it temp)

`delete_value(int value)`

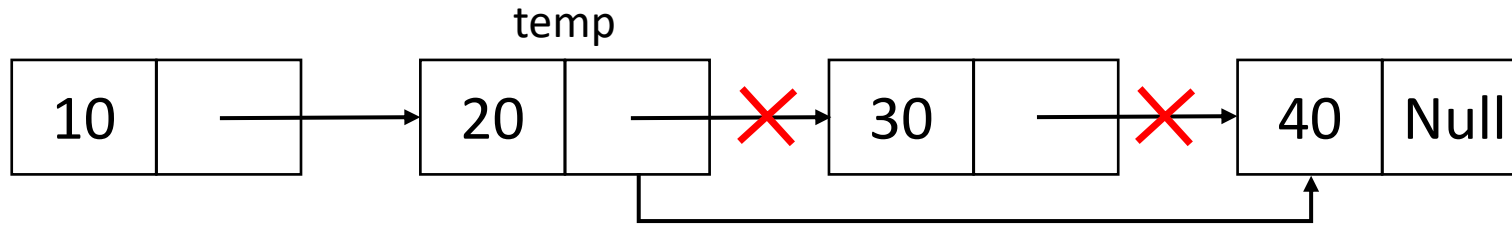


Delete node with
value 30

- Find the node **for**
which the next
node contains the
target value (Lets
call it temp)

20->next = 40
? = ?

`delete_value(int value)`



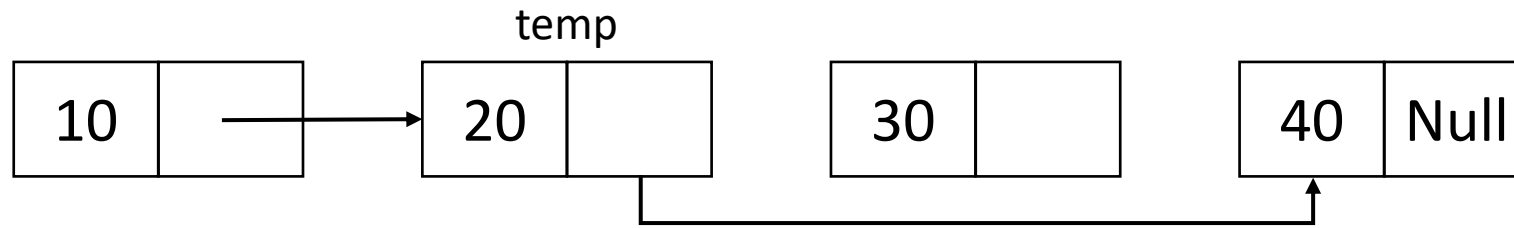
Delete node with
value 30

- Find the node **for which the next node contains the target value** (Lets call it temp)

`20->next = 40`

`temp->next = temp->next->next`

`delete_value(int value)`



Delete node with
value 30

- Find the node **for which the next node contains** the target value (Lets call it temp)

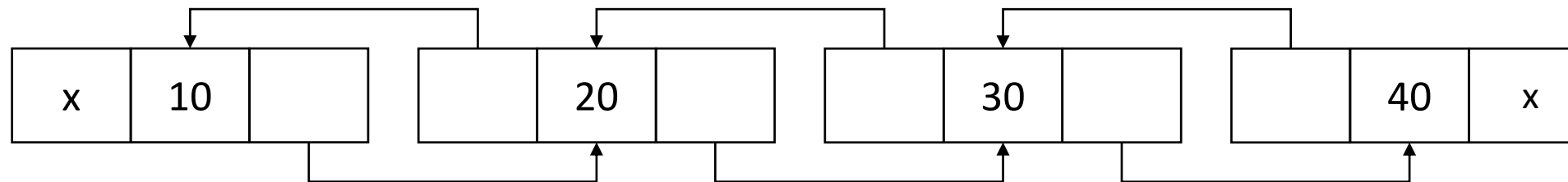
`20->next = 40`

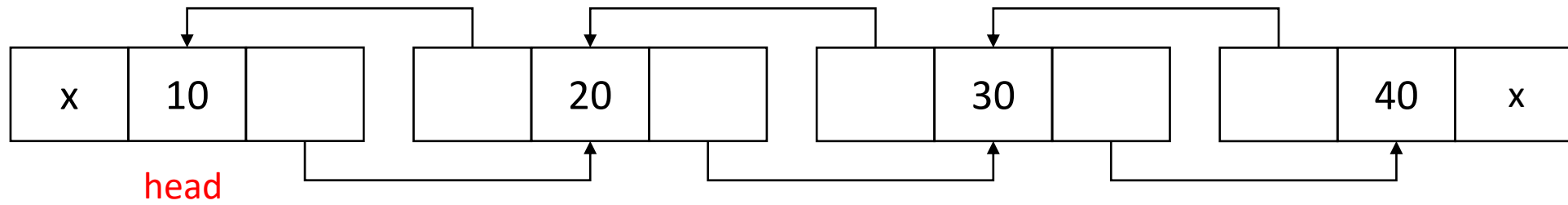
`temp->next = temp->next->next`

```
delete_value(int value){  
    if head->data == value:  
        head = head->next  
    else:  
        struct node* iterator = head  
        while iterator->data != value:  
            temp = iterator  
            iterator = iterator->next  
        if iterator != Null  
            temp->next = temp->next->next  
}
```

Doubly Linked lists!

- *Allows efficient traversal in both directions.*
- *Each node contains a pointer to the previous node and a pointer to the next node*



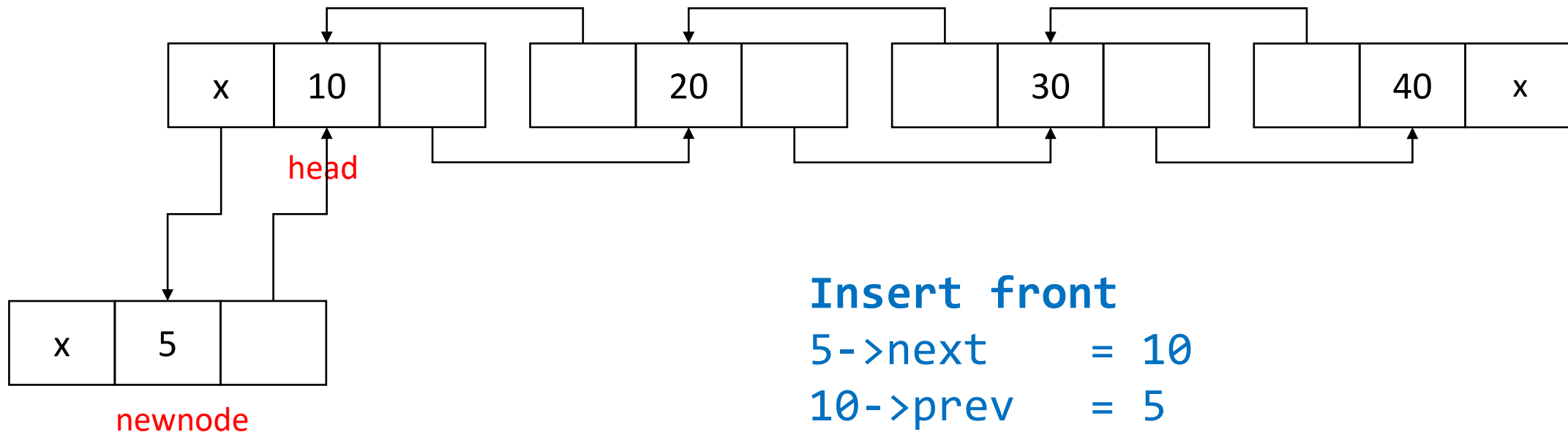


newnode

Insert front

Connect 5 with 10

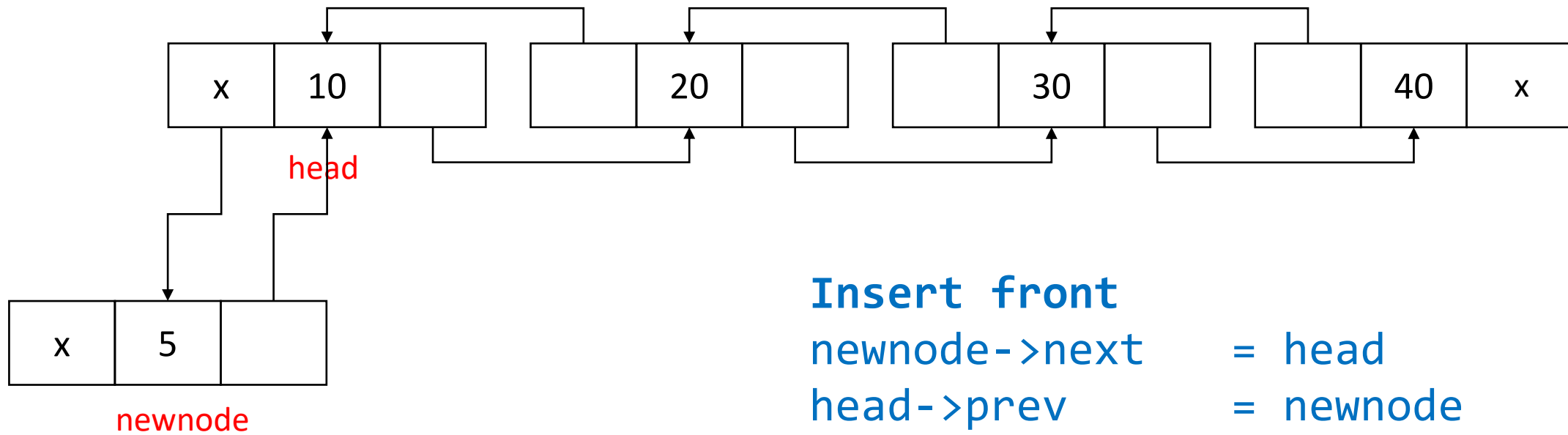
Connect 10 with 5

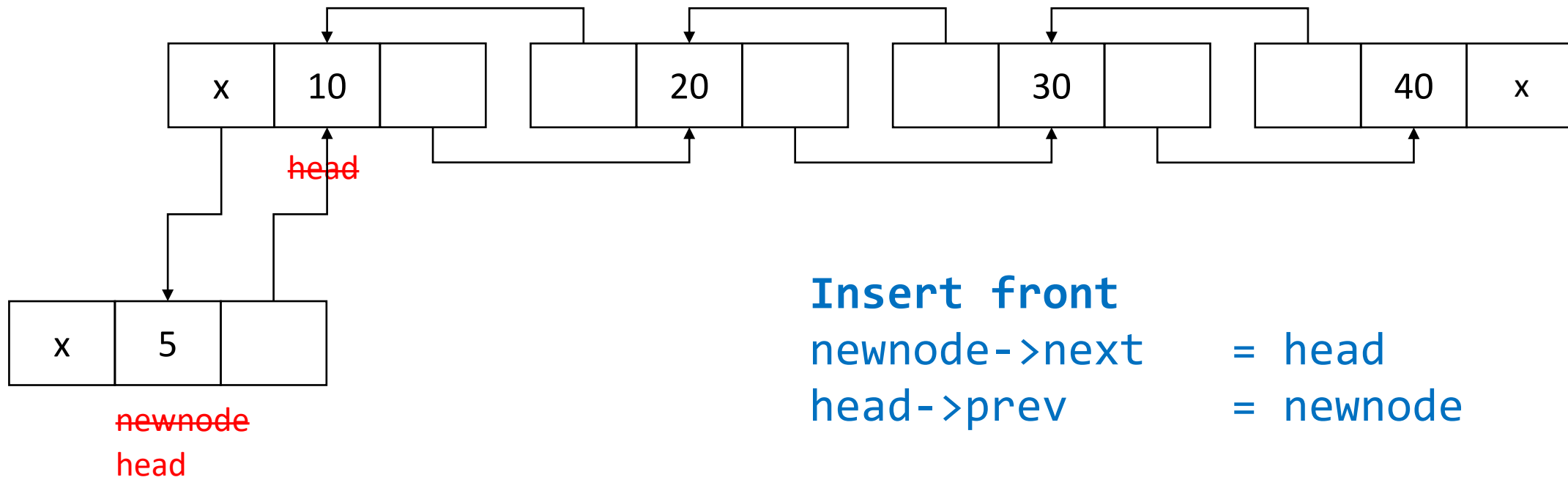


Insert front

5->next = 10

10->prev = 5

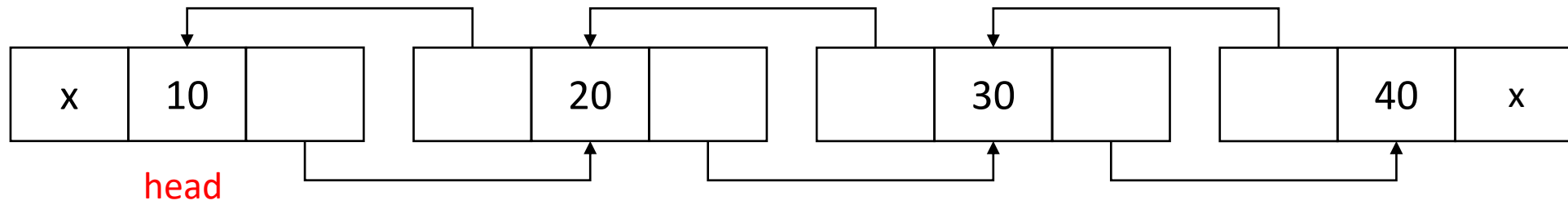




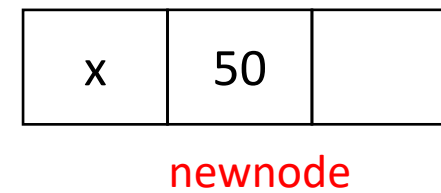
Insert front

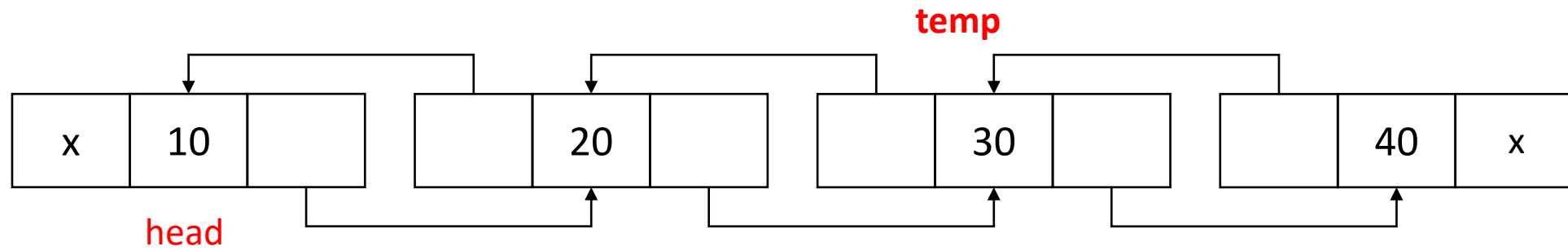
newnode->next = head
head->prev = newnode

head = newnode

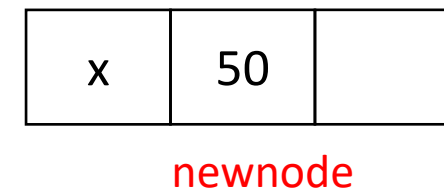


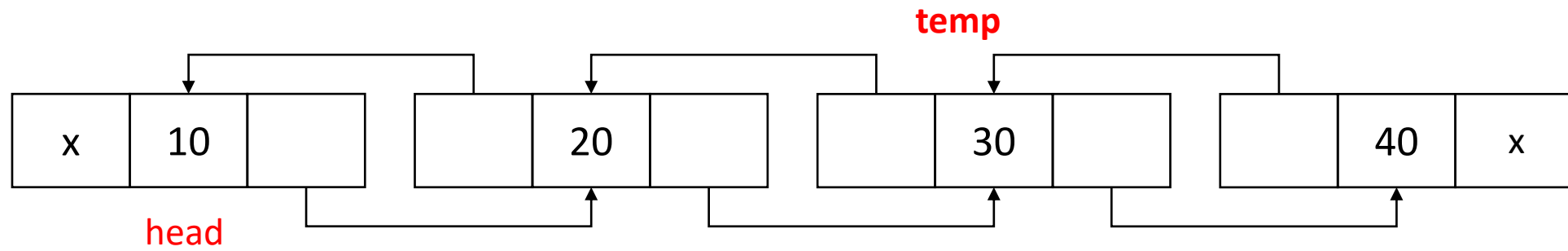
Insert after value (30)



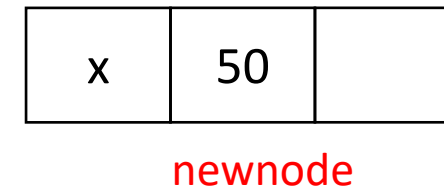


Insert after value (30)
-Find the node with the
desired value (temp)

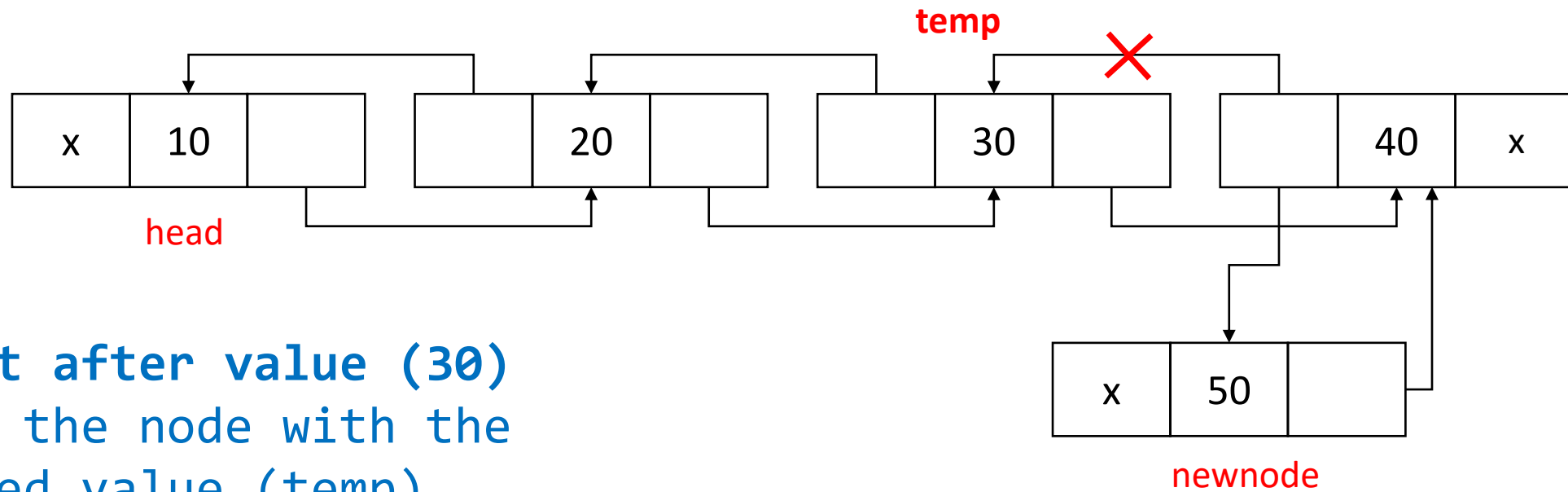




Insert after value (30)
 -Find the node with the
 desired value (temp)



Connect 50,40
 Connect 30,50

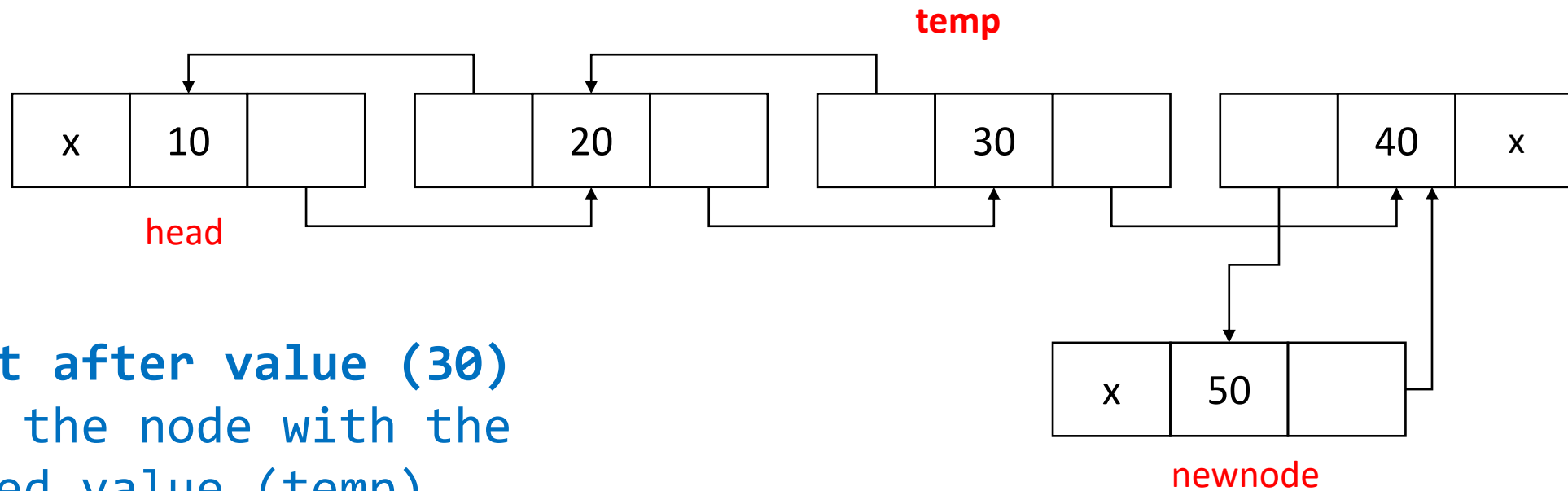


Insert after value (30)

-Find the node with the desired value (temp)

Connect 50,40: 50->next = 40
 40->prev = 50

Connect 30,50

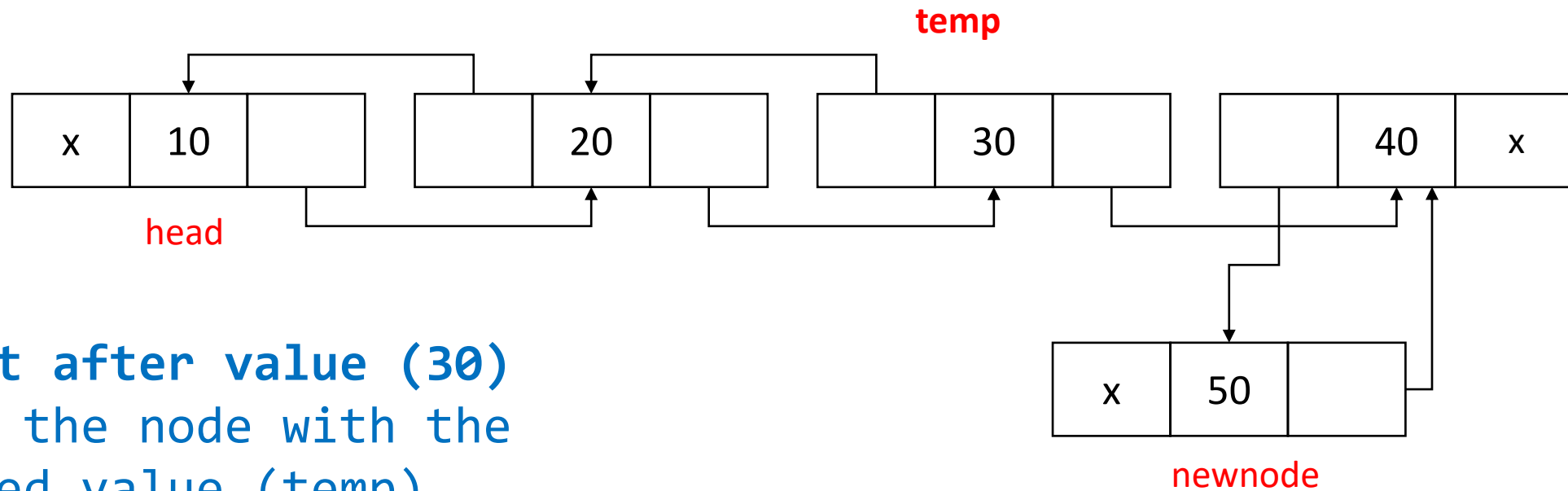


Insert after value (30)

-Find the node with the desired value (temp)

Connect 50,40: newnode->next = 40
 40->prev = newnode

Connect 30,50

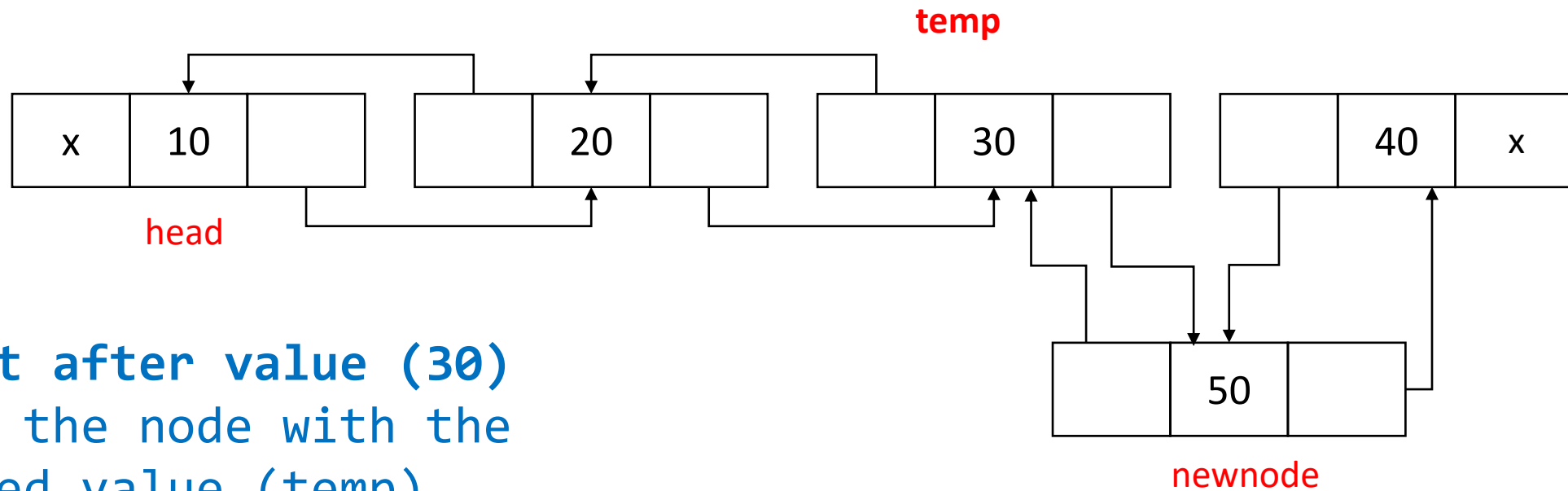


Insert after value (30)

-Find the node with the desired value (temp)

Connect 50,40: $\text{newnode} \rightarrow \text{next} = \text{temp} \rightarrow \text{next}$
 $\text{temp} \rightarrow \text{next} \rightarrow \text{prev} = \text{newnode}$

Connect 30,50

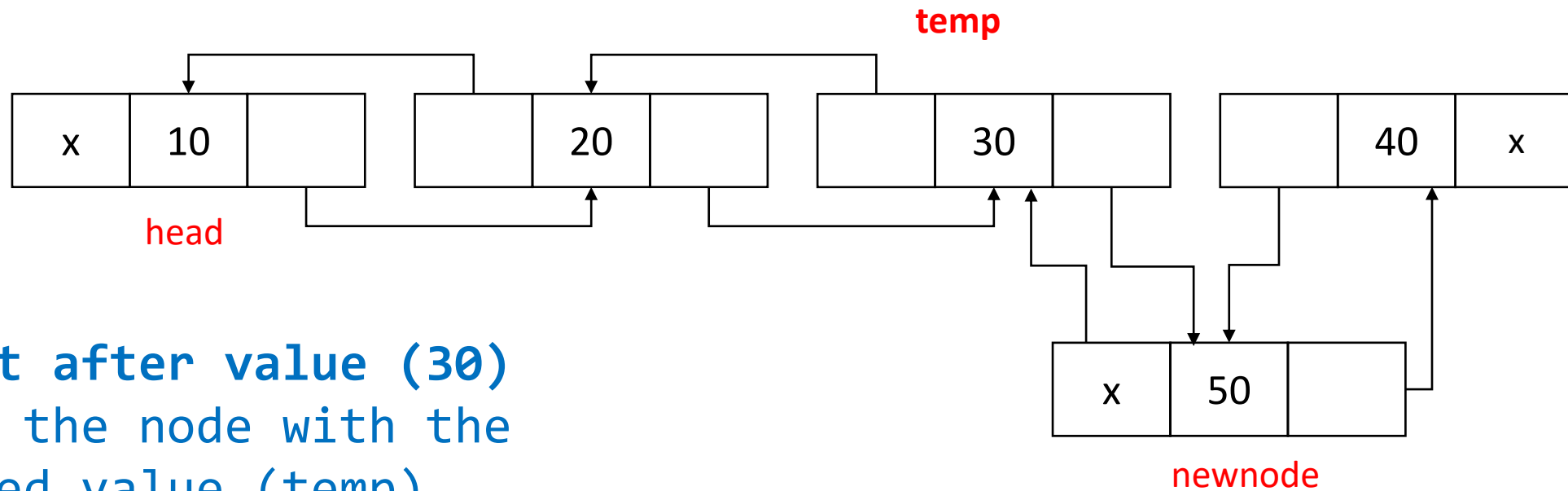


Insert after value (30)

-Find the node with the desired value (temp)

Connect 50,40: $\text{newnode} \rightarrow \text{next} = \text{temp} \rightarrow \text{next}$
 $\text{temp} \rightarrow \text{next} \rightarrow \text{prev} = \text{newnode}$

Connect 30,50: $30 \rightarrow \text{next} = 50$
 $50 \rightarrow \text{prev} = 30$

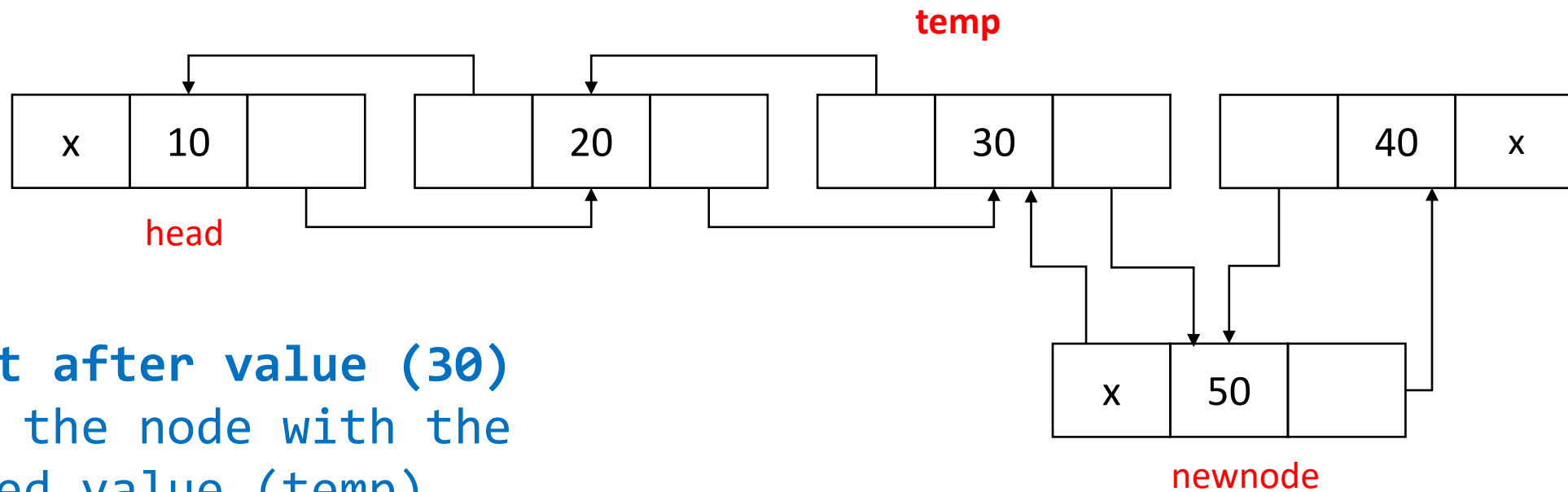


Insert after value (30)

-Find the node with the desired value (temp)

Connect 50,40: $\text{newnode} \rightarrow \text{next} = \text{temp} \rightarrow \text{next}$
 $\text{temp} \rightarrow \text{next} \rightarrow \text{prev} = \text{newnode}$

Connect 30,50: $30 \rightarrow \text{next} = \text{newnode}$
 $\text{newnode} \rightarrow \text{prev} = 30$

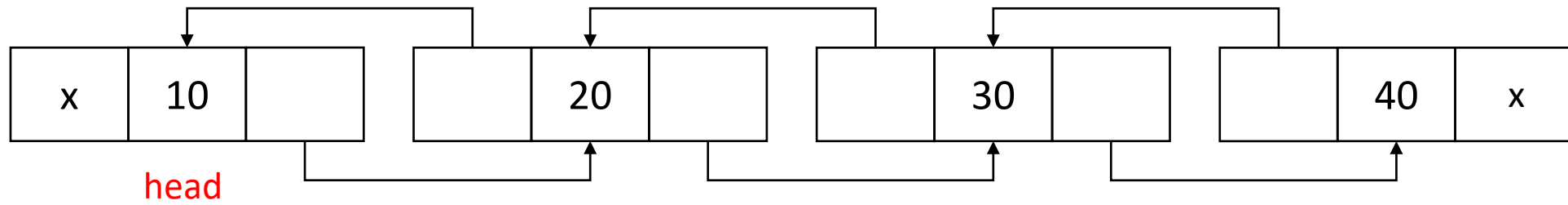


Insert after value (30)

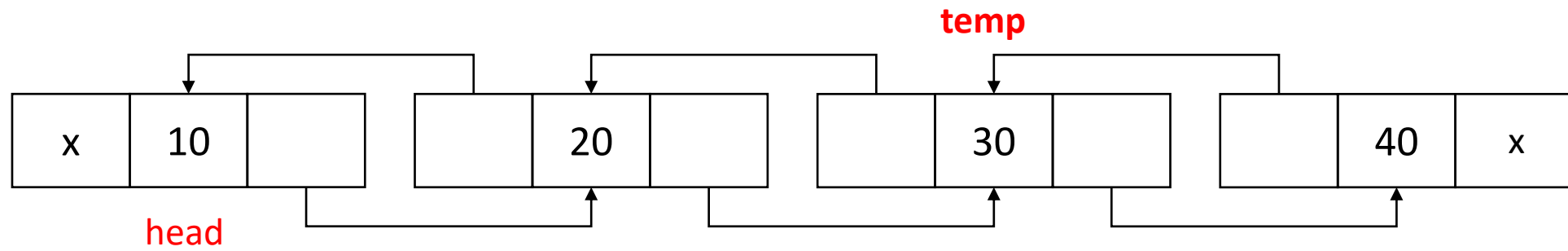
-Find the node with the desired value (temp)

Connect 50,40: `newnode->next = temp->next`
 `temp->next->prev = newnode`

Connect 30,50: `temp->next = newnode`
 `newnode->prev = temp`

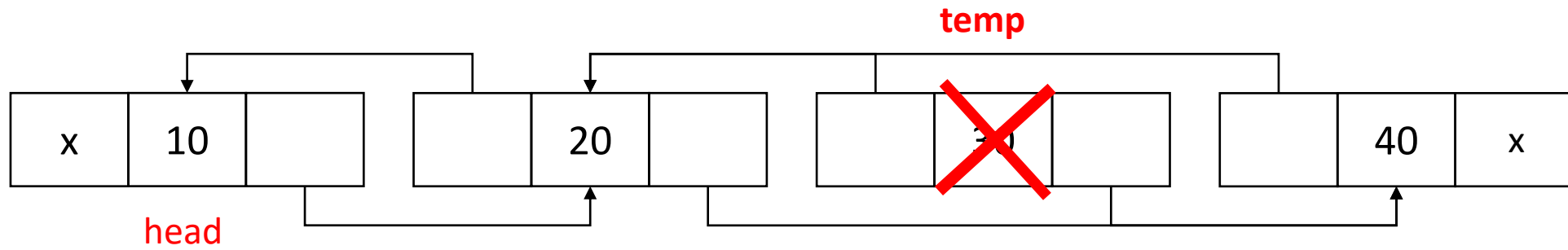


Delete node with value (30)



Delete node with value (30)

First, find the node containing the desired value (temp)

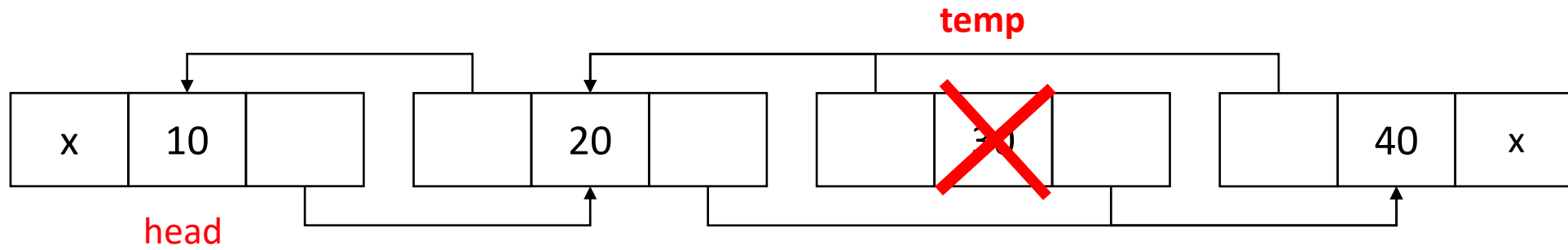


Delete node with value (30)

First, find the node containing the desired value (temp)

20->next = 40

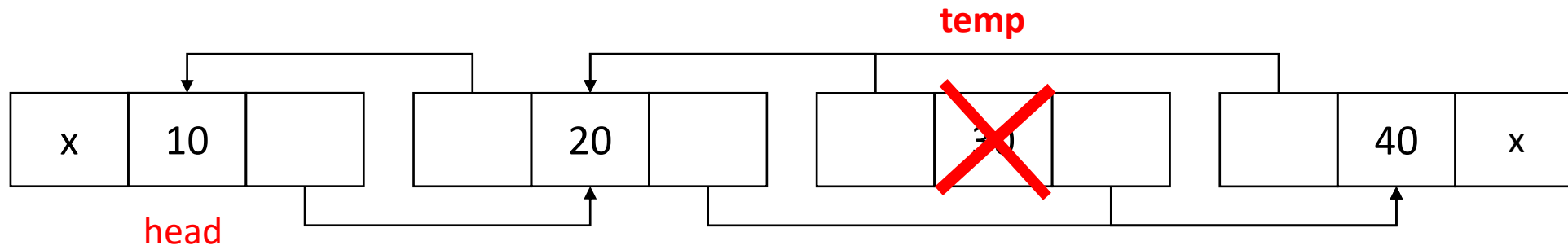
40->prev = 20



Delete node with value (20)

First, find the node containing the desired value (temp)

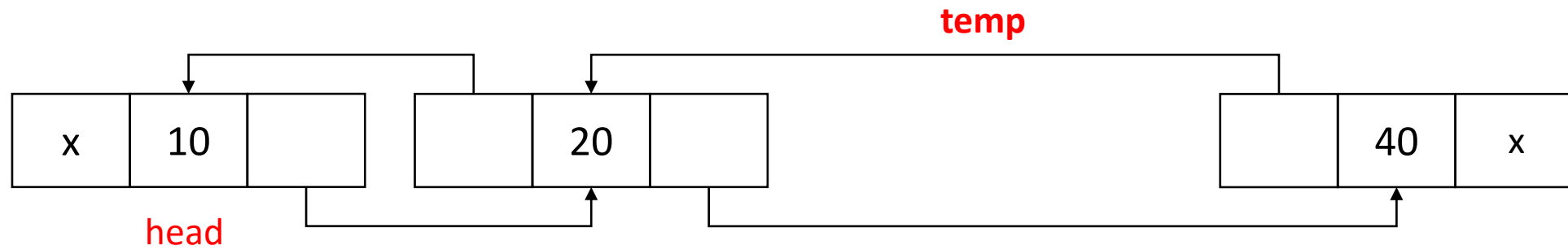
temp->prev->next = 40
40->prev = temp->prev



Delete node with value (20)

First, find the node containing the desired value (temp)

$\text{temp} \rightarrow \text{prev} \rightarrow \text{next} = \text{temp} \rightarrow \text{next}$
 $\text{temp} \rightarrow \text{next} \rightarrow \text{prev} = \text{temp} \rightarrow \text{prev}$



Delete node with value (20)

First, find the node containing the desired value (temp)

`temp->prev->next = temp->next`
`temp->next->prev = temp->prev`

References

Thank you